Code Guru

Team Setup Guide

How to get all four services running on your own machine, now that they share one login, one set of student data and one interface.

1. What you are setting up

Code Guru is four services in separate repositories: **Code Coach**, **Study Guider**, **PairPath** and the **Adaptive Gamification Engine**. All four are integrated against the same login.

The one thing to understand before anything else: **Code Coach owns login for the entire platform.** It is the only service that checks passwords. Study Guider and PairPath do not have their own accounts - they receive a token that Code Coach issued and ask Code Coach whether it is valid. There is a single web login screen, the **CodeGuru Portal**, which lives in the code-coach repo.

This means **you must run Code Coach locally**, even if you are only working on Study Guider or PairPath. Without it, every authenticated route in your service returns 503. The good news is that Code Coach needs no configuration at all to run locally - see section 5.

Every student record now lives in Code Coach. Its shared deployment stores accounts, sessions, diagnostics and remediation triggers in **Firestore**. Your service still owns its own domain data - PairPath its pair sessions in Postgres, the Gamification Engine its questions and game sessions in MongoDB, Study Guider its quiz history in Neo4j - but none of them owns *who the student is* any more.

> **Your old login no longer works**
> PairPath used to keep its own passwords, the Gamification Engine had its own user table, and Study Guider had a student id hardcoded in App.jsx. All three are gone. Whatever credentials you were testing with before will now be rejected, and that is correct behaviour, not a bug.
>
> Register once through the portal at `http://localhost:4200` and use that account everywhere. One account, all four services.

Ports

Service	Port	Who needs it
Code Coach backend	`8000`	Everyone. The identity provider.
CodeGuru Portal (login UI)	`4200`	Everyone, unless you use your own dev-login page.
Study Guider backend	`8010`	Study Guider devs.
Study Guider frontend	`5173`	Study Guider devs.
PairPath API	`3001`	PairPath devs.
PairPath frontend	`3000`	PairPath devs.
PairPath ml-service	`8020`	PairPath devs, only for live pair sessions.
Gamification API	`3002`	Gamification devs.
Gamification frontend	`5174`	Gamification devs.

Service	Port	Who needs it
Gamification ml-service	`5000`	Gamification devs, for difficulty prediction.
VS Code sign-in callback	`53682`	Nothing to start. The extension opens it briefly.

Port 8000 is taken by Code Coach
PairPath's ml-service README says to run it on port 8000. Do not. That is Code Coach's port. Start the ml-service on **8020** and make sure `ML_SERVICE_URL` in `Pair_Path/api/.env` points at 8020 - its built-in default is 8000, which would send ML requests to Code Coach.

2. Before you start

Check you have these. Versions below are what the team is using.

Tool	Check with	Needed for
Node.js 20+	`node --version`	All frontends, PairPath API, the portal
Python 3.12+	`python --version`	Code Coach, Study Guider, ml-service
Git	`git --version`	Everything
Java 17+ (optional)	`java -version`	Only to run Code Coach's VS Code extension

3. Get the code

You already have your own repository. What is new is that **everyone also needs code-coach** - it is the identity provider, so nothing in your service authenticates without it running.

You are the...	Already have	Also need to clone
Study Guider dev	Study-Guider	code-coach
PairPath dev	Pair_Path	code-coach
Gamification dev	adaptive-gamification-engine	code-coach

You have write access to your own repository only, which is all you need - nobody is expected to push to a service they do not own. code-coach is cloned to **run**, not to change. If you find a bug in it, tell the Code Coach developer rather than opening a branch there.

```
# Put code-coach beside your existing repo, in the same parent folder.
cd <the folder that already contains your repo>

git clone https://github.com/R26-SE-036/code-coach.git
```

Then bring your own repo up to date. Every repo does its work on the **dev** branch - never commit to main.

```
cd <your repo>   && git checkout dev && git pull && cd ..
cd code-coach    && git checkout dev && git pull && cd ..
```

Your .env files come separately, by message

A `.env` is gitignored in every repo, so pulling never brings one and never overwrites one. That is deliberate: they hold real database URLs and API keys. It also means the integration settings your service now needs **cannot reach you through git**.

You will be sent the merged .env files privately. They are your original settings - your own database, your own API keys, unchanged - **plus** the handful of keys integration added. Replace your existing files with them.

Read this before you replace anything
The files you are sent were merged from the .env you gave us, so nothing of yours was dropped. But keep a copy of your current .env anyway - takes two seconds, and if a value did get lost in transit you will want it back rather than regenerated.

Do NOT copy `.env.example` over a real .env. The examples carry placeholder values, and overwriting a .env with one destroys credentials that exist nowhere else.

```
cd code-coach    && git checkout dev && git pull && cd ..
cd Study-Guider  && git checkout dev && git pull && cd ..
cd Pair_Path     && git checkout dev && git pull && cd ..
cd adaptive-gamification-engine && git checkout dev && git pull && cd ..
```

Pulling later
Code Coach changes affect everyone, because it owns the login and the API contract. Run `git pull` in **code-coach** before you start work each day, not just in your own repo.

4. Two ways to run it

Pick one. Most of the time you want Mode A.

	Mode A - everything on your machine	Mode B - shared Code Coach via Cloudflare
When	Daily development. Building your own features.	Testing the real integration together, or demoing.
Code Coach	You run your own on localhost:8000.	One person runs it and shares a tunnel URL.
Accounts	Yours only, in memory. Gone when you restart it.	Shared, in Firestore. Everyone sees the same students and triggers.
Setup	No credentials, no accounts, no tunnel.	Tunnel owner needs the Firebase key; you need their URL.
Cloudflare	Not needed at all.	Required.

In Mode A, Code Coach falls back to **in-memory storage** when it finds no Firebase credentials. That is exactly what you want while building: no setup, no shared state to corrupt, and a clean slate on every restart. It prints `Storage backend: in-memory` at startup so you always know which mode you are in.

What Mode A costs you
Because storage is in-memory, every account and every remediation trigger is lost when you restart Code Coach - including the account you registered in the portal. Register again, or re-run the seed tool (section 14), which takes about ten seconds.

This is also why your teammates cannot see your test data in Mode A, and why the numbers on Home start at zero every morning. If you want data that persists and is shared, that is Mode B.

Already have a .env? Do NOT copy over it
Every service in this project keeps a `.env` that is gitignored, so it never travels with the code - it holds real database URLs and API keys that only exist on your machine. **Copying .env.example over it destroys those values.**

If you already have a .env: open the .env.example beside it and ADD only the keys you are missing. The examples are written as supersets - they list the original settings as well as the ones integration added - so a key-by-key comparison is all it takes. Copy the whole file only when there is no .env there at all.

5. Code Coach - everyone needs this

Backend

```
cd code-coach/backend

python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000
```

No .env file is required for Mode A. Leave it absent and Code Coach uses in-memory storage. Confirm the startup line says `Storage backend: in-memory`. Then check `http://127.0.0.1:8000/health` returns `{"status":"ok"}`.

The portal (the login screen)

```
cd code-coach/portal

npm install
copy .env.example .env          # ONLY if you have no .env yet - see section 3

npm run dev                     # http://localhost:4200
```

File to edit: `code-coach/portal/.env`

Variable	Mode A (local)	Mode B (Cloudflare)
VITE_CODE_COACH_URL	http://127.0.0.1:8000	the shared tunnel URL
VITE_ALLOWED_REDIRECTS	http://localhost:5173,http://localhost:3000,http://localhost:5174	same
VITE_STUDY_GUIDER_URL	http://localhost:5173	same
VITE_PAIRPATH_URL	http://localhost:3000	same
VITE_GAMIFICATION_URL	http://localhost:5174	same
VITE_ENABLE_DEV_LOGIN	true	true
VITE_VSCODE_LOOPBACK_PORT	53682	same

`VITE_ALLOWED_REDIRECTS` must also contain `http://127.0.0.1:53682`, which is where the VS Code extension listens while you sign in through the browser. It is a fixed port precisely because this list is checked by exact origin - a random port could never be allowed.

Adding a new service? It needs TWO entries here
A service missing from `VITE_ALLOWED_REDIRECTS` is refused at sign-in with *"That return address is not allowed"*, and one missing its `VITE_*_URL` simply never appears on the hub - so there is no link to click and no error explaining why. Both were missed when the Gamification Engine was added.

VITE_ALLOWED_REDIRECTS is a security control, not a convenience
It is the list of addresses the portal is allowed to hand your login token back to. Without it, anyone could link you to the portal with their own address and collect your token the moment you signed in. If the portal says *"That return address is not allowed"*, add your origin to this list - do not disable the check.

6. Study Guider

Backend (port 8010)

```
cd Study-Guider/backend

python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env          # ONLY if you have no .env yet - see section 3

.\.venv\Scripts\python.exe -m uvicorn app.main:app --port 8010
```

File to edit: `Study-Guider/backend/.env`

Variable	Mode A (local)	Mode B (Cloudflare)	Required?
`CODE_COACH_URL`	http://127.0.0.1:8000	the shared tunnel URL	Yes
`CORS_ORIGINS`	leave the default	leave the default	Yes
`NEO4J_URI`	your Neo4j Aura URI	same	Only to save quiz progress
`NEO4J_PASSWORD`	your Neo4j password	same	Only to save quiz progress
`OPENROUTER_API_KEY`	your OpenRouter key	same	Only for AI lessons

Login and remediation work **without** Neo4j and without an OpenRouter key. You will see `"database": "Disconnected"` on `/api/health`, the Analytics tab will say "No Data Points Found", and lessons will use fallback text instead of AI-generated content. That is fine for building UI. Ask the team for the real values when you need them.

If the backend crashes instantly with UnicodeEncodeError
This is a known bug: `app/services/ml_service.py` prints emoji, which the default Windows console cannot encode. Work around it by setting `PYTHONIOENCODING=utf-8` before running (PowerShell: `$env:PYTHONIOENCODING="utf-8"`).

Frontend (port 5173)

```
cd Study-Guider/frontend

npm install
copy .env.example .env          # ONLY if you have no .env yet - see section 3

npm run dev                     # http://localhost:5173
```

File to edit: `Study-Guider/frontend/.env`

Variable	Mode A (local)	Mode B (Cloudflare)
`VITE_API_BASE_URL`	http://127.0.0.1:8010	http://127.0.0.1:8010 (still local)
`VITE_CODE_COACH_URL`	http://127.0.0.1:8000	the shared tunnel URL
`VITE_PORTAL_URL`	http://localhost:4200	http://localhost:4200 (run your own)
`VITE_ENABLE_DEV_LOGIN`	true = own login form; false = use the portal	same

VITE_ENABLE_DEV_LOGIN decides which login you see
Set to **true**, Study Guider shows its own small login form and never touches the portal - faster while building. Set to **false** (or delete the line), it redirects to the portal, which is the real production flow. If you are testing the portal handoff and nothing happens, this is why. Vite only reads .env at startup, so restart after changing it.

7. PairPath

PairPath needs a PostgreSQL database. The team uses a free **Neon** cloud database - no local install, and it survives reboots. Ask for the connection strings.

API (port 3001)

```
cd Pair_Path/api

npm install
copy .env.example .env          # ONLY if you have no .env yet - see section 3

npx prisma generate
npx prisma migrate dev          # creates the tables, first time only

npm run start:dev               # http://localhost:3001
```

File to edit: `Pair_Path/api/.env`

Variable	What to put	Required?
`DATABASE_URL`	Neon DIRECT connection string. Used by migrations only.	Yes
`DATABASE_URL_POOLED`	Neon POOLED string, plus &pgbouncer=true on the end. Used at runtime.	Yes
`JWT_SECRET`	Any long random string. Must be the same for the whole team if you share a database.	Yes
`CODE_COACH_URL`	Mode A: http://127.0.0.1:8000 / Mode B: the tunnel URL	Yes
`ML_SERVICE_URL`	http://127.0.0.1:8020 (NOT 8000 - that is Code Coach)	Live sessions only
`MONGODB_URI`	Leave blank. Logs an error and continues.	No
`REDIS_URL`	Leave blank. Falls back to an in-memory store.	No

Do not change the Prisma versions
`prisma` and `@prisma/client` must both stay on **5.22.x**. If npm upgrades them, the API dies at startup with `Cannot find module '@prisma/client/runtime/library.js'`. Fix: set both back to ^5.22.0 in package.json, delete node_modules/@prisma, then `npm install && npx prisma generate`.

Frontend (port 3000)

```
cd Pair_Path/frontend

npm install
copy .env.local.example .env.local   # ONLY if you have no .env.local yet

npm run dev                     # http://localhost:3000
```

File to edit: `Pair_Path/frontend/.env.local`

Variable	Mode A (local)	Mode B (Cloudflare)
`NEXT_PUBLIC_CODE_COACH_URL`	http://127.0.0.1:8000	the shared tunnel URL
`NEXT_PUBLIC_PORTAL_URL`	http://localhost:4200	http://localhost:4200
`NEXT_PUBLIC_ENABLE_DEV_LOGIN`	true = own login form; unset = use the portal	same

> **The filename must start with a dot**
> Next.js reads `.env.local`. A file named `env.local` without the leading dot is silently ignored - your settings will look applied and will not be. This has already caught someone on this team.

ml-service (port 8020) - only for live pair sessions

```
cd Pair_Path/ml-service

python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe -m uvicorn app.main:app --port 8020
```

You do not need this for login, the dashboard, or session history. Start it only when you are testing the live collaboration features that call the ML model.

8. Adaptive Gamification Engine

Three parts: an Express API backed by MongoDB, a React frontend, and a Flask service that predicts game difficulty with a Random Forest. It has no accounts of its own - it verifies every request against Code Coach and reads the student's struggle data from there.

Backend API (port 3002)

```
cd adaptive-gamification-engine/backend

npm install
copy .env.example .env          # ONLY if you have no .env yet - see section 3

npm start                       # http://localhost:3002
```

File to edit: `adaptive-gamification-engine/backend/.env`

Variable	Mode A (local)	Mode B (Cloudflare)	Required?
`CODE_COACH_URL`	http://127.0.0.1:8000	the shared tunnel URL	Yes
`PORT`	3002	3002	Yes
`MONGODB_URI`	mongodb://localhost:27017/code-guru	same	Yes - games and questions
`ML_SERVICE_URL`	http://127.0.0.1:8030	same	Difficulty prediction only
`CORS_ORIGINS`	leave the default	leave the default	Yes

> **There is deliberately no JWT_SECRET here**
> This service does not issue or verify tokens itself - Code Coach does, via `GET /api/v1/auth/me`. An earlier version verified signatures locally with a shared secret, which meant signing out of Code Coach did nothing here. If you find code reaching for a signing secret, it is a leftover.

MongoDB is genuinely required for this service - unlike the others, its own data (question bank, game sessions, player profiles) lives there. Without it the API starts but the game routes fail. The dashboard still works, because struggle data comes from Code Coach.

Frontend (port 5174)

```
cd adaptive-gamification-engine/frontend

npm install
copy .env.example .env          # ONLY if you have no .env yet - see section 3

npm run dev                     # http://localhost:5174
```

File to edit: `adaptive-gamification-engine/frontend/.env`

Variable	Mode A (local)	Mode B (Cloudflare)
`VITE_CODE_COACH_ORIGIN`	http://127.0.0.1:8000	the shared tunnel URL
`VITE_GAMIFICATION_API_URL`	http://localhost:3002/api/v1	same
`VITE_PORTAL_URL`	http://localhost:4200	http://localhost:4200
`VITE_ENABLE_DEV_LOGIN`	true = own login form; false = use the portal	same

The frontend reaches Code Coach through a Vite proxy rather than calling it directly, so the browser stays same-origin and CORS never applies. `VITE_CODE_COACH_ORIGIN` is where that proxy forwards to - change that one, not the API path below it.

ml-service (port 5000)

```
cd adaptive-gamification-engine/ml-service

python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe app.py          # http://localhost:5000
```

If it is down the backend falls back to a simple rule (many unresolved errors or a low average score means an easier game), so the engine keeps working - but the difficulty you see is then a hand-written if/else, not the Random Forest. The response carries `fallback: true` when that happens.

9. What your service asks Code Coach for

These are the calls your code already makes. You do not have to wire them up - they are in place - but you should know what they are for, because breaking one shows up as a blank screen rather than an error.

All of them take `Authorization: Bearer <the student's access token>`. The `me` in each path means *the student this token belongs to* - there is no student id in any URL or body, and that is deliberate. Study Guider used to read the student id out of the request body, which let anyone read anyone else's progress by typing a different id.

Study Guider

Call	What it is for
`GET /api/v1/auth/me`	Verify the caller and learn who they are. Runs on every authenticated route, cached ~60s.
`GET /api/v1/remediation/me/recommendations`	The trigger cards on the home screen: concepts Code Coach watched the student get wrong repeatedly, already shaped into lesson + quiz + rationale.
`GET /api/v1/remediation/me/triggers`	The raw trigger list behind those recommendations.
`POST /api/v1/remediation/me/triggers/{id}/lesson-opened`	Fired when a micro-lesson is opened. Moves the trigger off pending.

Call	What it is for
`POST /api/v1/remediation/me/triggers/{id}/quiz-completed`	Fired with {quiz_id, score_percent}. 70% or more closes the trigger - this is what completes the remediation loop.
`GET /api/v1/dashboard/me/overview`	The Analytics tab: counts, mastery, concept trends and recent activity across all four services.
`GET /api/v1/dashboard/me/timeline`	A longer activity history than the overview carries.

Adaptive Gamification Engine

Call	What it is for
`GET /api/v1/auth/me`	Verify the caller. Every route in the Express API goes through this.
`GET /api/v1/students/me/struggling-concepts`	Which concepts to build a game around. This is the real diagnostic history - it replaced a local Mongo mirror that was only ever filled by a seed script.
`GET /api/v1/gamification/me/recommendations`	What practice Code Coach suggests next, including the game type.
`POST /api/v1/gamification/me/adaptation-decisions`	Reports the difficulty the engine chose and why, so the decision is on the student's record.
`POST /api/v1/gamification/me/session-results`	Reports the finished game - score, concept, difficulty actually played. This is what feeds the student's mastery and makes the game count for something.
`POST /api/v1/learning-sessions`	Opens a learning session so activity can be grouped over time.

PairPath

Call	What it is for
`GET /api/v1/auth/me`	Called inside POST /auth/exchange to verify the platform token before issuing a PairPath one. Fails closed: an unverifiable token is refused, never waved through.
`POST /api/v1/auth/login, /register`	The older password proxy. Still present only because the seeded demo pair has no Code Coach account. Not part of the normal sign-in path any more.

503 and 401 mean different things
401 - Code Coach answered and refused the token. The student needs to sign in again. **503** - Code Coach could not be reached at all. The student's login is fine; your service just cannot check it. Do not clear the session on a 503, or an outage logs everyone out.

10. PairPath keeps its own user rows - here is why

This section is mainly for the PairPath developer, but it explains the one place where the platform deliberately does *not* use Code Coach's id directly.

Every other service can simply carry the Code Coach access token around. PairPath cannot, for two concrete reasons:

What	Why it forces a local id
Foreign keys	Every row in PairPath's Postgres - pair sessions, session members, peer reviews, ML events - points at users.id. Swapping in a foreign id would orphan the lot.

What	Why it forces a local id
The Socket.IO handshake	It verifies PairPath's own JWT signature. It cannot validate a token Code Coach signed, and changing that is how you break live pair sessions.

So PairPath exchanges rather than adopts

`POST /auth/exchange` takes the Code Coach access token, verifies it with `GET /api/v1/auth/me`, then finds or creates the matching local row and issues **PairPath's own JWT**. Matching order matters:

#	Match on	Outcome
1	`users.codeCoachUserId`	Already linked. Reuse the row - all its history stays attached.
2	`users.email`	The account existed locally before integration. Link it by writing codeCoachUserId, rather than creating a duplicate.
3	`nothing matched`	First time here. Create the row and link it.

Two tokens live in the browser, and they are not interchangeable

localStorage key	Whose token	Used by
`token`	PairPath's own JWT	Every call to the PairPath API, and the Socket.IO handshake. This is the one that matters inside PairPath.
`codeguru.accessToken`	Code Coach's platform token	Kept so PairPath can call Code Coach on the student's behalf. Sending it to PairPath's own API will be rejected - and there is a test asserting that.

Do not 'simplify' this by deleting the local user table
It looks like duplication and it is not. `users.codeCoachUserId` is the join between the platform identity and PairPath's data; drop it, rename it, or point a foreign key at the Code Coach id instead, and every existing session, review and ML event loses its owner.

11. The shared interface

The four services used to look like four products - one was light with a purple accent and pastel backgrounds, the other three were dark, in two different typefaces. They now share one light blue-and-white theme and one navigation bar.

Where you start

`http://localhost:4200` is the whole platform's front door. Sign in there and you land on **Home**: a card per service, each showing a live number pulled from Code Coach (diagnostics, open remediations, pair sessions, games played), and a recent-activity feed showing which service did what. If the numbers are all zero you have a working setup and no data yet - run the seed tool in section 14.

The bar at the top of every service

Same bar, same position, in all four apps. Its links are the supported way to move between services, because the four run on four different origins and a plain link would arrive with no session. Each link goes through `{portal}/go?to=<service>`, which hands the token over on the way.

Moving between services by typing the URL will look broken
Typing `localhost:3000` straight into the address bar takes you to PairPath with no session, so it bounces you to the portal to sign in again. That is not a bug - browsers keep localStorage per origin. Use the bar.

Changing how it looks

File	What it controls
`portal/src/styles/codeguru-theme.css`	MASTER copy. Every colour, radius, shadow, easing and font for all four services, plus the bar's own styling.
`/src/styles/codeguru-theme.css`	A copy. Editing it changes only your service and puts you out of step with everyone else.

Colours are exposed twice: `--cg-rgb-*` holds raw 'R G B' triplets (which is what lets PairPath's Tailwind keep working with opacity modifiers like `bg-surface-800/50`), and `--cg-*` holds ready-made colours for stylesheets and inline styles. Use the token, never a hex code.

> **var() does not work everywhere**
> It does not resolve inside SVG presentation attributes. `stroke="var(--cg-accent)"` on a Recharts series or a lucide icon renders with no colour at all. Set CSS `color` and let `currentColor` do the work, or resolve the token in JS first - Study Guider's `src/lib/theme.js` does exactly that for its charts. The same applies to anything that draws its own SVG, like Mermaid and Monaco.

Dark mode is not built yet. `:root[data-theme='dark']` in the theme file is deliberately empty - because everything reaches colour through these tokens, filling it in is a values-only change plus a toggle in the bar. No component needs to be touched.

Signing in to the VS Code extension

Code Coach's extension no longer asks for a password in a VS Code prompt. Sign In opens the portal in your browser; when you finish, the extension picks the session up automatically over a loopback callback on port 53682. If that port is busy it says so and falls back to the old prompts.

12. What not to touch

None of these are style preferences. Each one has a specific failure attached, and most of them fail quietly - a blank panel, a wrong name, a silent 401 - rather than throwing something you would notice.

Leave alone	What breaks if you change it
The wire field names in codeguru-auth.js: identifier, password, client_name, full_name, email	They are the Code Coach API contract, not a naming style. Renaming any of them to camelCase makes the request fail validation.
codeguru-auth.js itself, in your repo	It is a copy. The master is in code-coach/portal/src/lib/. Edit it there and run sync-codeguru-auth.sh, or the four services drift apart.
codeguru-theme.css and CodeGuruBar in your repo	Also copies, synced by sync-codeguru-shared.sh. Change one and your service stops matching the others.
The codeguru.* localStorage keys	The portal hands a session over by writing exactly those keys. Rename one and the handoff silently writes past your app - it will look like login simply did nothing.
VITE_ALLOWED_REDIRECTS - never add a wildcard	It is the list of addresses the portal may hand a token to. Without it the portal is an open redirect that gives anyone's token to any URL that asks.
The ports in the table in section 1	They were chosen to stop collisions that already happened: the Gamification API was on 3000 (PairPath's frontend) and Study Guider's backend on 8000 (Code Coach).
users.codeCoachUserId in PairPath's schema	The join between platform identity and PairPath's own data. See section 10.
Reading a student id from a request body or URL	It comes from the bearer token. Anything else lets one student read another's data.

Leave alone	What breaks if you change it
Clearing the session on a 503	503 means Code Coach was unreachable, not that the token was bad. Clearing it logs everyone out over a blip.

13. Running it day to day

Open a terminal per service and start them in this order. Code Coach must be first - the others verify tokens against it.

Study Guider developers

#	Terminal	Command
1	code-coach/backend	`.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000`
2	code-coach/portal	`npm run dev`
3	Study-Guider/backend	`.venv\Scripts\python.exe -m uvicorn app.main:app --port 8010`
4	Study-Guider/frontend	`npm run dev`

PairPath developers

#	Terminal	Command
1	code-coach/backend	`.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000`
2	code-coach/portal	`npm run dev`
3	Pair_Path/api	`npm run start:dev`
4	Pair_Path/frontend	`npm run dev`
5	Pair_Path/ml-service	`.venv\Scripts\python.exe -m uvicorn app.main:app --port 8020`

If you set the dev-login flag to true in your frontend, you can skip terminal 2 (the portal) entirely.

Gamification developers

#	Terminal	Command
1	code-coach/backend	`.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000`
2	code-coach/portal	`npm run dev`
3	adaptive-gamification-engine/backend	`npm start`
4	adaptive-gamification-engine/frontend	`npm run dev`
5	adaptive-gamification-engine/ml-service	`.venv\Scripts\python.exe app.py # port 5000`

This service needs its MongoDB Atlas cluster reachable for the game routes.

Mode B only - the Cloudflare tunnel

One person runs Code Coach and exposes it. Everyone else points their `CODE_COACH_URL` at the printed address.

```
cloudflared tunnel --url http://localhost:8000
```

Who	What they do
Tunnel owner	Runs Code Coach with the Firebase key configured (so data persists and is shared). Adds everyone's browser origins to CORS_ALLOWED_ORIGINS in code-coach/backend/.env.
Everyone else	Puts the tunnel URL in CODE_COACH_URL, VITE_CODE_COACH_URL and NEXT_PUBLIC_CODE_COACH_URL, then restarts their dev servers.

The tunnel URL changes every restart
Cloudflare quick tunnels get a new hostname each time. When the tunnel owner restarts, everyone must update their .env files and restart their dev servers. This is why no URL is hardcoded anywhere in the codebase.

14. Getting test data

Study Guider shows nothing until Code Coach has raised a remediation trigger, and that only happens after the same mistake repeats three times. Rather than doing that by hand, run the seed tool:

```
cd code-coach/backend
.\.venv\Scripts\python.exe -m app.dev_tools.seed_student

# Mode B - seed against the shared backend instead:
.\.venv\Scripts\python.exe -m app.dev_tools.seed_student --base-url https://<tunnel-url>
```

It creates a student, generates three real struggles (array indexing, loop boundaries, conditional logic), and prints an access token you can paste into curl or Postman. Sign in as `seed.student@example.com / seed-student-1234`. Running it twice is safe - it reuses the account and creates no duplicates.

15. Check it actually works

#	Do this	You should see
1	Open http://127.0.0.1:8000/health	{"status":"ok"}
2	Open http://localhost:4200	The Code Guru sign-in screen, blue on white
3	Register a new account there	Home, with a card per service and a bar across the top
4	Use the bar to open your service	You arrive already signed in - no second login
5	Check the address bar after arriving	No access_token left in the URL
6	Check the name in the bar	Your name, and the same name in every service
7	Run the seed tool, reload Study Guider	Three struggle cards with real concept names
8	Reload Home	The counts moved - the services are writing to one student record

If step 4 sends you back to a login screen, your service's origin is probably missing from VITE_ALLOWED_REDIRECTS - see section 5.

16. Troubleshooting

Every one of these has actually happened to someone on this team.

Symptom	Cause	Fix
Your service returns 503 on every route	Code Coach is not running.	Start it on port 8000. Everything needs it.

Symptom	Cause	Fix
Portal says "That return address is not allowed"	Your origin is missing from the allow-list.	Add it to VITE_ALLOWED_REDIRECTS in code-coach/portal/.env, restart.
Study Guider shows its own login, not the portal	VITE_ENABLE_DEV_LOGIN is true.	Set it to false in Study-Guider/frontend/.env and restart Vite.
PairPath: "PairPath is not responding"	The PairPath API on 3001 is not running.	Start it. Your login is fine - click Try again after.
PairPath .env.local seems to be ignored	The file is named env.local without the dot.	Rename it to .env.local and restart.
Cannot find module '@prisma/client/runtime/library.js'	prisma and @prisma/client are on different majors.	Pin both to ^5.22.0, delete node_modules/@prisma, npm install, npx prisma generate.
UnicodeEncodeError on Study Guider startup	Emoji in print() vs the Windows console encoding.	Set PYTHONIOENCODING=utf-8 before running.
uvicorn.exe: "Fatal error in launcher"	The venv was created before the folder moved. Venvs are not relocatable.	Delete .venv, recreate it, reinstall requirements. Or use python -m uvicorn.
VS Code extension: no yellow underlines, no error	You are signed out. Auto-analysis fails silently by design.	Ctrl+Shift+P, "Code Coach: Sign In".
ML requests behave strangely in PairPath	ml-service is on 8000, colliding with Code Coach.	Run it on 8020 and set ML_SERVICE_URL to match.
Gamification dashboard is empty for a student who has struggles	No triggers yet, or you are signed in as a different student.	Run the seed tool (section 14) and sign in as that student.
Gamification game routes fail but the dashboard works	MongoDB is down. Only this service truly needs it.	Start MongoDB on 27017.
Gamification API returns 503 on every route	It cannot reach Code Coach to verify your token.	Check CODE_COACH_URL and that Code Coach is running. 503 means unreachable, 401 means the token was refused.
Your old test login is rejected	Identity moved to Code Coach. Per-service accounts are gone.	Register once at http://localhost:4200 and use that account everywhere.
Signing in from VS Code says "Not Found"	Your Code Coach backend predates the handoff endpoints.	Restart it. Python does not reload without --reload.
The bar shows the wrong person's name	A stale profile in localStorage from an earlier account.	Pull code-coach and your repo - this was fixed in codeguru-auth.js. Or clear site data.
A page element is hidden behind the top bar	It is position:fixed or sticky at top-0.	Anchor it at top: var(--cg-bar-h) instead of 0.
Sign-in feels slow the first time	A brand-new token misses the auth cache and Code Coach must reach Firestore.	Expected. Roughly a second. Later calls with the same token are cached and near-instant.

17. Rules of the road

Rule	Why
Work on the dev branch. Never commit to main.	main is the reviewed branch. All three repos follow this.
Never commit a .env file.	They hold database passwords and API keys. Every repo already gitignores them - keep it that way.
Do not edit codeguru-auth.js in your own repo.	It is a copy. The master lives in code-coach/portal/src/lib/. Change it there and run code-coach/sync-codeguru-auth.sh, or the three services drift apart.
Never read the student id from a request body.	It comes from the bearer token. Trusting the body lets anyone read anyone else's data - Study Guider had exactly this bug before integration.
Pull code-coach before you start work.	It owns the login and the API contract, so its changes affect everyone.
Do not edit codeguru-theme.css or CodeGuruBar in your own repo.	Copies, like codeguru-auth.js. The master is in code-coach/portal, synced by sync-codeguru-shared.sh.
Never hardcode a colour. Use a --cg-* token.	A hex code cannot follow the theme, and will be the one thing on screen that looks wrong when dark mode lands.
Treat a 503 from Code Coach as temporary, not as a bad login.	401 means the token was refused. 503 means Code Coach was unreachable - clearing the session there logs everyone out over a blip.

Full API reference: `code-coach/integration/README.md` and `API_CONTRACT.md` in the same folder, which is generated from the running service and therefore cannot drift from the code.